



Prince Sultan University

College of Computer and Info Sciences / Department of Computer Science

Term 252

CYS402 : Secure Software Development

Section : 1407

Canvas (LMS)

Prepared by:

Joud AlNitaifi - 221410121

Haifa Alangari - 221410567

Nora Alomier - 222410172

Haya Albakr - 219410618

Instructor:

Dr. Zainab Masood

Table of Contents :

Overview of the Software Domain

- Introduction to the Software

- GitHub Source Code Repository

- Domain Context

- Importance of Security in This Context

Identification of Significant Security Risks

Data Protection

Consequences of Security Failures

- Immediate Consequences

- Long-Term Consequences

- Impact on Stakeholders

Table of Team Contribution

Overview of the Software Domain

Canvas is a web-based learning management system (LMS) designed to support and help online teaching and learning for educational institutions including schools, universities and colleges. It

offers a platform where instructors can manage their courses easily and upload assignments and quizzes and track students progress. Students can also use canvas to submit their assignments , view their grades, and take online quizzes .Canvas is a cloud based application that supports web and mobile access , making it scalable and accessible to large academic communities. Due to the sensitive nature of data handled by Canvas , security is important throughout its development process and usage .

GitHub Source Code Repository

<https://github.com/instructure/canvas-lms>

Online Documentation Link:

<https://canvas.instructure.com/doc/api/>

Domain Context

Canvas belongs to the field of educational technology and uses an academic information system for online learning environments. It plays an important role in supporting learning activities in universities and schools by providing them with digital tools for managing courses and students' performance .

Key Features:

- **Managing and creating course content for instructors.**
- **Conducting online quizzes**
- **Submitting assignments and receive feedback digitally**
- **Providing communication tools such as announcements and discussion boards .**

Main stakeholders:

The primary users involved in Canvas include :

- **Students who access learning courses and submit academic work**
- **Instructors who manage courses and evaluate student performance**
- **Academic administration who review and oversee systems operations and policies**
- **External service providers that integrate learning tools with the platform**

Importance of Security in This Context

Security is a critical aspect of Canvass because it directly affects student privacy, academic honesty and institutional reliability. Any weaknesses in the system could lead to unauthorised access to personal information , manipulation of grades . In addition universities and schools are required to

follow data protection and privacy regulations such as FERPA-like policies and general data protection standards.

Identification of Significant Security Risks

Because it helps run key parts of schoolwork, Canvas LMS faces serious threats. Weak spots in code open doors - flaws built into how things are set up matter too. Outside forces often target systems like these, especially ones people access through browsers. How features work together can create unseen gaps. Attackers look for any break in defenses, not just obvious holes.

1. Data Breaches

Risk Description:

Breach risks rank high when it comes to Canvas LMS. Sensitive details flow through the system - names, test results, login codes, school histories. When weak spots pop up in software or linked tools, outsiders might slip in. Unauthorized eyes could see what should stay hidden. Exploited flaws open doors that were meant to stay shut.

Origins of the risk:

A faulty check on user data opens doors to attacks like SQL injection. When websites fail to inspect inputs closely, risks creep in through methods like cross-site scripting. Weak oversight invites trouble of various kinds. Mistakes in handling form entries lead straight into dangerous territory. Poor validation acts like a welcome sign for hackers. Flaws appear where checks should have been strict. Gaps in scrutiny allow harmful code to slip through unnoticed.

A single weak link in an app's outside connections can open the door. Keys left lying around online often tie back to live systems. When shortcuts skip security checks, gaps appear without warning. Hidden access points sometimes show up in shared tools. Mistakes in setup may let strangers step right through.

Backups sitting unprotected because settings were left open by mistake. One wrong step when setting limits lets data slip through gaps. Exports saved without locks invite trouble later on. Mistakes in who can see what often lead straight to exposed records.

Older versions of public code libraries left without updates can create openings. Where fixes exist but aren't applied, weaknesses remain exposed. Gaps like these invite unwanted access. When shared tools run on stale builds, risks grow silently. Unmaintained modules become backdoors by neglect. Exposure increases when patches sit unused. Flaws linger where maintenance stops.

Potential Impact:

When school records get seen by the wrong people, private details might slip out. Hacked information can damage honesty in learning environments. Someone else could pretend to be a student later on. Rules meant to guard personal info may end up broken.

2. Unauthorized Access**Risk Description:**

Someone steps into areas they should not reach when permissions are bypassed. Hitting a Canvas LMS account without approval might expose grades, test answers, lessons, or work turned in by students. Access like that skips the rules meant to keep things locked down.

Origins of the risk:

Many people pick passwords that are easy to guess. Some repeat the same password across multiple accounts. Logging in often depends on just one step. Extra verification steps are rarely required. Security improves when more checks exist. Simple habits create big risks. Adding a second confirmation method helps block intruders.

Credential theft through phishing or social engineering attacks.

Too many users get more access than they need because roles are set up wrong.

Vulnerabilities in session handling that enable session hijacking.

Potential Impact:

Faking grades messes up trust in schoolwork. When students cheat, it shakes faith in what people really know. Changing materials without permission leads to doubts about accuracy. The system begins feeling less solid when rules get bent.

3. System Outages and Service Disruption**Risk Description:**

When a system goes down, students might miss deadlines. Outages block access to lessons, stop exam attempts, disrupt homework uploads - education stumbles without steady uptime. Keeping platforms running matters more than it seems at first glance.

Origins of the risk:

Distributed Denial-of-Service (DDoS) or Denial-of-Service (DoS) attacks targeting application servers or APIs.

Mistakes in setup might trigger cloud outages. When systems misfire, services often halt unexpectedly. Configuration slips usually expose hidden weaknesses. Problems like these disrupt operations without warning.

Glitches in code might shake things loose. Unchecked patches can rattle the system's balance. When fixes aren't fully checked, hiccups creep in. Rough edges show when testing falls short. Hidden flaws emerge after rollout. Untested changes sometimes spark wobbles. Stability cracks under unproven tweaks.

Finding shortcuts by overloading tools on purpose. Running hidden scripts that push limits beyond normal use.

Potential Impact:

When lessons get cut short, school routines stumble. Missed due dates pile up without warning. Tests fall apart midstream. Classrooms face daily hiccups that slow everything down.

4. Data Integrity Attacks**Risk Description:**

Fairness in learning depends on unbroken data within Canvas LMS. When bad actors change scores or tamper with submissions, confidence erodes - trust doesn't survive silent edits. What stands firm one day might shift overnight unless safeguards hold.

Origins of the risk:

Changing grades has poor checks in place. What happens is oversight slips through. One mistake opens the door to bigger issues. Security gaps show up when actions matter most. Loopholes remain because systems trust too easily. Risk grows where verification should be tight.

Few records kept when risky actions happen. Watched too little during critical tasks.

Insider threats or the compromise of instructor or administrator accounts.

Potential Impact:

When grades get questioned, trust in schools slips away. Mistakes linger, shaping how institutions are seen years later. Problems today echo far into the future

Data Protection

Types of Data Managed by Canvas (Sensitive Data)

The system also includes very important academic information for example student grades, assignment submissions, quiz and exam attempts, and instructor feedback on the attempts and their grades . This data is very personal because it reveals students' performance in their tasks and assessments. No unauthorized person should be able to view or change grades or feedback, as this can affect students academically and personally.

Canvas also contains important course information like lecture materials, discussion files, shared documents, and other course resources. Many instructors consider this content their own work and do not want it to be shared outside their courses or accessed by students who are not enrolled.

There is also sensitive system related data that users usually are not authorized to see it , such as login session information, tokens, and password reset data. If this type of data is exposed, an attacker could use it to access user accounts without approval.

Canvas keeps logs and audit records that track system activities. These records are important for security and accountability because they help identify what happened if a problem or issue came up. This data must be protected and stored properly so it can be used when needed.

Existing Security Measures for Data Protection

Canvas has many security controls to protect the system and the data inside it. These controls help students and instructors use the system comfortably and feel that their work and information are safe.

First, every user must log in before using Canvas. This is important because it makes sure that only authorized users, such as students, instructors, and administrators, can access the system. Without logging in, no one can use the features or see any data.

Canvas manages user roles. The system only grants access in accordance with each user's stated role. such as ,students cannot edit grades or change other students' submissions, they are only authorized to do what students are allowed to perform such as submitting assignments and check

instructor feedback on their attempts .Instructors are allowed to manage grades and course materials. This control helps keep the system fair and prevents users from doing things they are not authorized to do.

Canvas also protects data while it is being sent between the user and the system. When information like grades or submissions is transferred, it is sent through secure connections such as HTTPS. This helps make sure the data is protected while moving and prevents anyone from interfering with it or changing it during transmission.

Session protection is another control used in Canvas. If a user stays inactive for a long time or forgets to close the system, the session will automatically end. This is important, especially if someone uses Canvas on a shared device, because it reduces the chance of another person accessing the account.

Canvas also keeps audit logs that record important actions inside the system, such as logins, grade changes, and other activities. These logs help with accountability because users know that their actions are recorded. This can prevent misuse, since students and instructors are aware that inappropriate actions can be traced back to them. Audit logs are also useful if a problem happens, because administrators can review what occurred and take the correct action.

Canvas also uses access tokens and API keys when it connects with other systems or tools. These tokens allow systems to exchange data without asking for a password every time. Because these tokens can give access to user data, they must be protected carefully. If a token is exposed, it could be misused, so Canvas needs to control how these tokens are created, stored, and revoked to keep the system secure.

Weaknesses and Areas for Improvement

security controls does not prevent vulnerabilities to be exploited there are still potential vulnerabilities that may affect data protection if not mitigated.

Unauthorized access is possible if users use weak passwords or reuse passwords across many apps or systems if there isn't multi-factor authentication enforced because if the password is obtained by an attack attack could use the same password to access other platforms. Also, wrong role or permission configurations could allow users to access data they are not authorized to access or view, such as other student grades or hidden course materials .

Sensitive information such as grades, exam questions, or reports could be exposed if sharing settings are misconfigured or access controls are not applied correctly, some users don't know they should turn on this setting so they don't apply it properly, educational and awareness are also very important to aware people about the controls they have to implement. API access

tokens used for system integrations can harm the system if they are kept in unsafe places or transmitted in insecure communication without encryption .

Data integrity is very important in learning management systems. If the system is compromised, attackers could attempt to modify grades, submissions, or timestamps, which would directly affect academic integrity.

Web application vulnerabilities also have a risk. As a web-based system, Canvas may be attacked for example , cross-site scripting or injection attacks if input validation and security updates are not properly managed and applied to the control mechanisms.

more secure password policies and multi-factor authentication should be used for all user types, user roles and permissions should be regularly reviewed ,private information should be encrypted when saved; log files ought to be reviewed for odd behaviors; and regular security testing and system updates should be carried out in order to improve data protection.

Consequences of Security Failures

Security problems in a Learning Management System like Canvas can have real and serious effects on everyone who uses the system, including students, instructors, and the institution itself.

Immediate Consequences

One of the most serious outcomes of a security breach is when unauthorized users gain access to sensitive information. This may include student personal details, grades, academic records, or course materials. When this happens, there is a risk of grades being changed, private information being exposed, or accounts being misused. If login systems or access tokens are compromised, attackers may also be able to act as legitimate users without being detected.

Another immediate issue is system disruption. If the platform or one of its connected applications is attacked or misused, students and instructors may temporarily lose access to important resources such as assignments, exams, or learning materials. This can interrupt classes, delay submissions, and negatively affect the overall learning experience.

Long-Term Consequences

If security problems continue to happen, trust in the system will slowly decrease. Students and instructors may begin to worry about whether their data is really protected, which can make them less willing to use the platform for important academic tasks. When trust is lost in this way, it is

difficult to rebuild, and this can affect how often the system is used and how satisfied users are in the long run.

Security breaches can also cause problems beyond the system itself. If private or academic data is exposed, institutions may have to deal with complaints, legal consequences, or even fines. On top of that, handling a security incident usually takes a lot of effort. Time and resources are spent investigating what went wrong, informing affected users, and making changes to prevent the same issue from happening again, which can place a heavy burden on the institution.

Impact on Stakeholders

- **Students:** Their personal information may be exposed, academic records could be altered, and their privacy may be violated.
- **Instructors:** Course integrity may be affected, grades could be changed without authorization, and teaching activities may be disrupted.
- **Institutions or Organizations:** They may experience financial losses, legal responsibility, damage to their reputation, and interruptions to normal operations.

Overall, security failures in Canvas-related software can weaken the reliability and effectiveness of digital learning systems. This shows how important it is to apply strong security practices to protect users and ensure a safe learning environment.

Table of Team Contribution

Team Member Name	Assigned Task	Reviewer
Haifa Alangari	Overview of the Software Domain	Joud Alnitaifi
Haya Albakr	Identification of Security Risks	Haifa Alangari

Nora Alomier	Data Protection	Haya Albakr
Joud Alnitaifi	Consequences of Security Failures	Nora Alomier

**College of Computer and Info Sciences - Department of Software
Engineering**

Term 252

Canvas LMS

Prepared by:

Joud AlNitaifi - 221410121

Haifa Alangari - 221410567

Nora Alomier - 222410172

Haya Albakr - 219410618

Instructor:

Dr. Zainab Masood

Table of Contents

1. Product Assets.....	3
1.1 Listing and Discussion of Product Assets.....	3
a. Data Assets (Most Critical).....	3
b. System Assets.....	6
c. Intangible Assets.....	7
1.2 Assets Prioritization.....	8
1.3 Decision Process.....	8
1.4 Future Changes.....	9
2. Malicious Actors and Threats.....	9
2.1 Types of Attackers.....	9
1. External hackers.....	10
2. Insider students.....	10
3. Malicious Administrators.....	10
4. Organized Cybercriminal Groups.....	11
2.3 STRIDE Threat Analysis.....	11
i. Spoofing.....	11
ii. Tampering.....	12
iii. Repudiation.....	12
iv. Information Disclosure.....	13
v. Denial of Service.....	13

vi. Elevation of Privilege.....	13
2.3 CIA Threat Distribution.....	14
2.4 Exploitation Methods.....	14
3. Software Requirements.....	15
3.1 Agree on Definitions.....	15
3.2 Identify Security Goals.....	15
3.3 Elicit Security Requirements.....	15
3.4 Prioritization of requirements.....	16
4. Team Contribution.....	17
References.....	18

1. Product Assets

A product asset is any resource (technical or non-technical) that has value to the organization and may be targeted by attackers. Identifying assets is the first step in structured risk analysis.

1.1 Listing and Discussion of Product Assets

a. Data Assets (Most Critical)

These properties are very sensitive information that Canvas LMS stores, processes, or transmits. Since the platform operates in an academic setting, data confidentiality and integrity are critical to maintaining trust, regulatory compliance, and operational stability. Breach of these assets may result in significant legal, academic, and reputational harm.

i. Student Personal information

The personal information of students includes student identification numbers, names, email addresses, and contact information, as well as enrollment records. This information is very sensitive because it uniquely identifies people and is frequently linked to other institutional systems. Such information may be used against students and faculty in the event of identity theft, phishing, or social engineering attacks if it is not kept secret. Moreover, privacy laws, such as the Family Educational Rights and Privacy Act (FERPA) and other data protection laws, protect students' personal information (EDUCAUSE, 2023). Confidentiality principles would be directly violated by any unauthorized disclosure, which could lead to legal consequences and loss of institutional credibility.

ii. Academic Records (Grades, Submissions, Feedback)

Academic history contains quiz attempts, exam results, assignment submissions, and teacher comments and grades. Such records are the foundational academic product of the learning management system and are directly linked to students' academic status and achievement. Their protection is important because grades determine scholarship and graduation eligibility, as well as prospects (OWASP Foundation, 2023). Any alteration of academic records made without the appropriate authorization would be against the principle of integrity of the CIA triad and might seriously harm trust in institutions. Even slight interference can give rise to disputes, appeals, or even legal proceedings, and this is one of the system's most important assets.

iii. Exam and Quiz Content

The material used in exams and quizzes includes question banks, assessment templates, and correct answer keys set up by instructors. This is intellectual property, confidential in nature,

and on which a fair evaluation of the academic work is to be conducted. By leakage, students can gain an unfair advantage before an assessment, thereby causing academic dishonesty and undermining evaluation procedures. Such exposure would not only jeopardize confidentiality and integrity but also tarnish the fairness and credibility of the academic system (EDUCAUSE, 2023). Recurring leaks may cause significant damage to the institution's image and undermine trust in online learning platforms.

iv. Authentication Credentials

Authentication records consist of password hashes, session tokens, password reset tokens, and multi-factor authentication (MFA) secrets. These credentials regulate access to system privileges and user accounts. When compromised, attackers can execute account takeovers, escalate privileges, or impersonate legitimate users. Since authentication systems serve as the first line of defense against unauthorized access, their breaches can result in large-scale data exposure across multiple assets. Securing these credentials is thus critical in ensuring confidentiality, integrity, and general system security.

v. API Tokens and Integration Keys

To facilitate secure communication between the LMS and other external systems, such as third-party educational tools, mobile applications, and automated data synchronization services, API tokens and integration keys are employed. These tokens enable systems to identify themselves without having to type in passwords. Nonetheless, when API tokens are exposed or improperly stored, an attacker might bypass conventional login systems and obtain direct system access (OWASP Foundation, 2023). Also, compromised tokens may enable massive automated threats, such as data scraping or bulk data retrieval. Given the growing popularity of system

integrations, API credentials are highly sensitive for safeguarding confidential information and preventing bad actors from gaining access to systems.

vi. Audit logs

Audit logs are rich documentation of system utilization, including login attempts, grade adjustments, permission adjustments, and administrative interventions. These records are necessary for accountability, incident response, and forensic investigations. Audit logs help administrators know what happened, when, and which accounts were involved in a security incident. Altering or deleting audit logs makes it challenging to audit malicious activities or demonstrate mischief, undermining non-repudiation and integrity controls (EDUCAUSE, 2023). Hence, audit logs should be securely stored and not tampered with in any form to ensure transparency and uphold the confidence that the system has security measures in place.

b. System Assets

System assets are the technical infrastructure components that support the functionality of Canvas LMS. System assets, unlike data assets, are the information stored in the system, as well as the hardware, software, and cloud environments that facilitate platform functionality. These elements are essential because, when compromised, attackers can control the entire application environment, resulting in serious breaches of confidentiality, integrity, and availability.

i. Web Application Servers

Web application servers host the application logic and provide the user interface through which students, instructors, and administrators interact with the LMS. They handle login requests, session administration, submission receipt, and backend operations. Web servers are widely used as gateways to the system, making them a popular target for injection, remote code

execution, and distributed denial-of-service (DDoS) attacks (EDUCAUSE, 2023). In the event of a compromise, attackers can run malicious software on the server, steal sensitive information, disrupt application functionality, or turn off services. Breaches of this level can result in massive data loss or the total interruption of academic operations, and web application servers are an extremely valuable system resource.

ii. Database Servers

Structured academic and personal information, such as student records, grades, course content, authentication data, and audit logs, is stored on database servers. They are the most prized pieces of the backend infrastructure, as they hold the core information that underpins the entire LMS ecosystem (OWASP Foundation, 2023). Cases of unauthorized access to the database might enable attackers to alter grades, destroy records, or seize large amounts of confidential student information. Databases are very likely to be compromised through SQL injection, privilege escalation, or misconfigured exploits, since they are central repositories of sensitive data. Database compromise would directly affect all three CIA principles: confidentiality, integrity, and availability, and its protection is crucial to the overall system security.

iii. Cloud Infrastructure

The cloud architecture comprises virtual machines, storage buckets, network components, load balancers, and other supporting services that host and scale the LMS environment. Since Canvas is a cloud-based platform, it relies heavily on well-configured cloud resources to ensure performance, scalability, and availability. Nevertheless, a key security risk is the misconfiguration of cloud services, including, but not limited to, publicly exposed storage buckets, improperly secured virtual machines, or loose network access controls (Cloud Security

Alliance, 2023). Attackers often use cloud misconfigurations to compromise systems or perpetrate large-scale data breaches. Since cloud infrastructure underpins the entire operating environment, its security configuration directly affects the system's resilience and reliability. Proper access controls, encryption, and continuous monitoring must therefore secure these assets.

c. Intangible Assets

Although non-technical, intangible assets are among the most valuable elements of Canvas LMS. These are institutional reputation, academic integrity, user trust, and regulatory compliance. Institutional reputation is the image of the university or organization in the eyes of students, faculty, and external stakeholders, and this image can be severely tarnished after a security breach. Academic integrity is the fairness and reliability of grading and assessment procedures, which can be compromised when grades or exam content are altered. User trust is key to further adoption and the successful use of the platform, since students and instructors should have confidence that their information and educational activities are secure. Compliance with regulations ensures compliance with the laws and policies that regulate student data protection, including the Family Educational Rights and Privacy Act (OWASP Foundation, 2023). These assets are not physical or digital systems, but they tend to suffer the greatest long-term losses following a security incident, and the trust and reputation are likely to require years to recover.

1. Secure User Authentication

→ **Specific:** Implement Multi-Factor Authentication (MFA) for all users (students, instructors, administrators) accessing Canvas LMS.

- **Measurable:** Achieve 100% adoption of MFA among users within 6 months.
- **Achievable:** Integrate MFA into the existing authentication system using an app-based method (e.g., Google Authenticator).
- **Relevant:** MFA will enhance security and prevent unauthorized access to sensitive student and academic data.
- **Time-bound:** Complete the integration and testing of MFA within 3 months.

2. Data Encryption at Rest

- **Specific:** Ensure all sensitive data (grades, personal information, session tokens) is encrypted when stored in the database.
- **Measurable:** Complete encryption of all data stored in the system, including academic records, by the end of the next quarter.
- **Achievable:** Use industry-standard encryption techniques such as AES-256 and ensure proper key management practices are followed.
- **Relevant:** Protects user data from unauthorized access and ensures compliance with privacy laws like FERPA.
- **Time-bound:** Implement encryption for all sensitive data within 3 months.

3. Audit Logging and Monitoring

- **Specific:** Implement a comprehensive audit logging system to track all significant actions (e.g., grade modifications, access attempts).
- **Measurable:** Ensure that 100% of administrative and academic actions are logged with timestamps and user identification.

- **Achievable:** Use the built-in logging features in the Canvas LMS and enhance them with custom logging for critical actions.
- **Relevant:** Logs will provide transparency, help detect suspicious activities, and assist in compliance with regulatory requirements.
- **Time-bound:** Complete the logging system integration within 2 months.

4. API Security

- **Specific:** Secure all Canvas LMS API endpoints using OAuth 2.0, ensuring encrypted communication and secure access tokens.
- **Measurable:** Achieve 100% secure API access through OAuth by the end of the next semester.
- **Achievable:** Integrate OAuth 2.0 into all public API endpoints and enforce secure API token management.
- **Relevant:** Prevent unauthorized access and ensure third-party integrations are secure.
- **Time-bound:** Complete the OAuth 2.0 integration within 4 months.

5. Denial-of-Service (DDoS) Protection

- **Specific:** Implement DDoS protection mechanisms to ensure Canvas LMS remains operational under high traffic conditions.
- **Measurable:** Achieve 99.9% uptime during high-traffic scenarios or attacks.
- **Achievable:** Use services like Cloudflare or AWS Shield for automatic DDoS protection and rate-limiting.

→ **Relevant:** Maintains the availability of the platform and ensures students and instructors can access the LMS at all times.

→ **Time-bound:** Complete the DDoS protection setup within 2 months.

1.2 Assets Prioritization

Assets were ranked using:

- Impact on CIA triad
- Legal implications
- Operational disruption level
- Attack attractiveness
- Recovery complexity

Rank	Asset	CIA Impact	Justification
1	Academic Records	C & I	Directly affects academic integrity
2	Authentication Credentials	C	Enables full system compromise

3	Exam Content	C & I	Enables cheating & trust loss
4	Database Servers	A	The system collapses if unavailable
5	API Tokens	C	Allows automated abuse
6	Audit Logs	I	Needed for accountability

1.3 Decision Process

The prioritization process followed structured reasoning:

- i. Identify asset value.
- ii. Evaluate potential attacker motivation.
- iii. Estimate exploitation likelihood.
- iv. Measure potential impact severity.
- v. Consider regulatory penalties.
- vi. Consider the recovery cost.

1.4 Future Changes

Asset value is dynamic and changes over time as the system expands and introduces new technology with Canvas LMS. For example, AI-based grading systems can make sensitive datasets and high-value algorithms available. The proliferation of mobile access increases

exposure to session tokens and API messages across a wide range of devices and networks. Also, the increased use of third-party integrations increases the attack surface through external APIs and mutual authentication systems (EDUCAUSE, 2023). Migration to the cloud and infrastructure scaling can also create configuration risks and distributed resources that need to be safeguarded. In addition, the number of students is growing, meaning that personal and academic data is stored in significant volumes, which increases the overall impact of any potential breaches. These reasons justify periodically reviewing (at least annually) the asset identification and prioritization process as part of the Secure Software Development Life Cycle (SSDLC) to ensure that security controls keep pace with a system's changing risk environment.

2. Malicious Actors and Threats

2.1 Types of Attackers

It is important to understand the various categories of attackers to develop effective security controls for Canvas LMS. Attackers possess varying motivations, skill sets, and access abilities, each of which affects the threats they can present to the system.

1. External hackers

External hackers are strangers to the system who seek to hack into it without the organization permitting them. Their main intentions are usually data theft, financial gain, and ransomware launch. In other instances, cybercriminals might be interested in selling stolen student information on dark markets or in making ransom payments after encrypting institutional networks. Such attackers frequently possess technical expertise that enables them to exploit web application vulnerabilities, execute phishing attacks to steal credentials, and carry out brute-force or credential-stuffing attacks on login systems (Cloud Security Alliance, 2023). Due to the

accessibility of LMS via the internet, external hackers are always a major and ongoing threat to system confidentiality and availability.

2. Insider students

Insider students are legitimate users of the platform, but they may seek to abuse their authorization privileges for personal gain. They can be driven by the desire to earn better grades unmeritoriously, to access exam content before exams, or to change the time stamps of submissions. Unlike external attackers, inside students already have valid accounts, so they do not need to intrude at the first stage. The main threat from insider students is the misuse of access privileges and the potential for privilege escalation (OWASP Foundation, 2023). For example, a student can attempt to corrupt system requests, exploit access control vulnerabilities, or learn vulnerabilities that enable them to obtain restricted instructor-level information. Their technical skill levels may differ, but insider threats are especially harmful because they are perpetrated by members of the trusted user base.

3. Malicious Administrators

High-impact insider threat: Malicious administrators or instructors have high permissions in the system that grant them high impact. The reasons might be abuse of authority, personal conflict, retaliation, or unauthorized interference with academic records. Since administrators already have access to sensitive functions such as grade modification, user management, and system configuration, misuse of these privileges may lead to serious integrity and confidentiality breaches (International Organization for Standardization, 2022). The danger is heightened by the fact that administrative measures may be considered lawful only if effective logging and supervisory systems are employed. As a result, segregation of duties and robust auditing measures play a major role in reducing privileged-user risks.

4. Organized Cybercriminal Groups

Organized cybercriminal groups are threat actors that are well-funded and highly skilled, and they usually operate on a large scale. Their reasons are usually ransomware execution and massive credential gathering. Educational institutions are potential targets of such groups, as they handle valuable personal data and may have limited cybersecurity budgets relative to financial institutions (Cloud Security Alliance, 2023). Such attackers usually organize coordinated attacks, exploit zero-day vulnerabilities, or use sophisticated phishing tactics to compromise multiple accounts simultaneously. Organized groups, by virtue of their resources and expertise, can cause large-scale disruption, such as a total system outage, a mass data breach, or attempted financial extortion, and are therefore among the gravest threats to system availability and confidentiality.

2.3 STRIDE Threat Analysis

The STRIDE threat modeling framework is used to identify high-level threats in Canvas LMS systematically. STRIDE classifies the threats into six groups, namely Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege, all aligned with the CIA (Confidentiality, Integrity, Availability) security tenets.

i. Spoofing

Spoofing is an attacker taking the identity of a legitimate user or system component to obtain unauthorized access. Spoofing in the LMS can be achieved using stolen credentials obtained through phishing, credential stuffing, or brute-force attacks. Hackers might also steal active sessions by robbing the insecure networks or hacked devices of session cookies or tokens. After successfully impersonating the victim, the attacker can access sensitive academic data or even perform activities on the victim's behalf (OWASP Foundation, 2023). Spoofing primarily

breaches the confidentiality principle by granting unauthorized access to the secured data, and it may also result in additional breaches of integrity if the malicious activities are conducted using the obtained account.

ii. Tampering

Tampering is an unauthorized alteration of data or system components. In the LMS context, this can include adjusting grades, tweaking assignment submissions, adjusting course material, or manipulating audit logs to hide ill-intentioned actions. This leads to actions that directly undermine the integrity of the system, since there is no longer any way to trust data as accurate or authentic (Cloud Security Alliance, 2023). For example, if a student successfully alters grades or a hacker alters instructional information, the academic integrity of the institution is severely compromised. It requires powerful access controls, input validation, and logging to prevent and detect tampering.

iii. Repudiation

Repudiation is the act of a user rejecting the action committed by the system, and the system is unable to provide enough evidence that the action was indeed performed. In an educational institution, it might be a student refusing to admit that they edited a piece of work, or an administrator refusing to acknowledge the erasure of valuable logs or alterations to grades. It is hard to establish accountability with no secure logging, timestamping, and traceability mechanisms (European Union Agency for Cybersecurity, 2023). The main impact of repudiation on integrity and trust is that the system must ensure that everything is captured in a verifiable format. The establishment of secure audit logs, digital signatures, and non-repudiation will assist in preventing disputes and aid forensic investigations.

iv. Information Disclosure

Information disclosure is unauthorized access to confidential data. Within the LMS, it may be caused by massive data leakage, API token disclosure, or database content leakage due to misconfiguration or other vulnerabilities, such as SQL injection. Student personal data, academic records, and authentication credentials may be disclosed. This threat specifically violates the confidentiality principle of the CIA triad (International Organization for Standardization, 2022). The problem of information disclosure may result in identity theft, academic fraud, and regulatory violations, and may be considered one of the most dangerous threats to the system.

v. Denial of Service

Denial-of-Service (DoS) attacks aim to render the system inaccessible to authorized users. Attackers can cause a distributed denial-of-service (DDoS) attack in a cloud-based LMS by flooding servers with inappropriate traffic or by abusing resource-depletion vulnerabilities to consume system memory, CPU, or database connections. These interruptions may prevent learners from accessing coursework, submitting assignments, or taking online examinations (OWASP Foundation, 2023). This risk directly contravenes the CIA model's principle of availability. Since academics are highly dependent on continuous access to the system, it is crucial to ensure resilience against DoS attacks to avoid disruptions to education.

vi. Elevation of Privilege

Elevation of privilege is a process in which a user is granted access to permissions they were not initially assigned. Examples include a student using an access control vulnerability to gain instructor-level privileges or an instructor misusing their elevated privileges to gain administrative privileges. This kind of threat is normally caused by the malfunctioning of access control systems or incorrect authorization checks (International Organization for Standardization,

2022). Privilege elevation violates confidentiality and integrity, as attackers can access sensitive information and modify data. Such threats have to be prevented through strict role-based access control (RBAC), the application of the least privilege principle, and ongoing monitoring of permission changes.

2.3 CIA Threat Distribution

CIA	Examples	Consequences
Confidentiality	Data breach, Spoofing	Legal penalties
Integrity	Grade tampering	Academic distrust
Availability	DDoS	Educational disruption

2.4 Exploitation Methods

Canvas LMS attackers can use various formal and social engineering methods to exploit vulnerabilities in the system. For example, they can conduct SQL injection attacks by using improperly validated input fields to access or corrupt backend databases. XSS payloads can be inserted into discussion boards or user-generated content sections, stealing session cookies or redirecting site users to malicious websites. Broken access control mechanisms can also be used by attackers to bypass authorization checks and access restricted resources without authorization. Students or instructors can be targeted by phishing to steal their login credentials, which can

subsequently be used in credential stuffing attacks on the platform (Cloud Security Alliance, 2023). Moreover, attackers may steal authentication tokens from insecure browser storage or improperly configured cloud storage resources, including publicly accessible buckets, to access sensitive data. These exploitation techniques underscore the importance of secure code development, strong authentication policies, and effective cloud configuration controls.

Malicious Actors, Threats, Mitigation & Skill Levels

#	Threat Actor	Main Threat Type	STRIDE Category	Targeted Asset	CIA Impact	Recommended Mitigation	Skill Level
1	External Hackers	Credential theft (phishing, brute force)	Spoofing	Authentication Credentials	Confidentiality	Multi-Factor Authentication (MFA), Account lockout, Strong password policy	Medium
2	External Hackers	SQL Injection	Tampering / Information Disclosure	Database Servers	Confidentiality & Integrity	Input validation, Prepared statements, Web Application Firewall (WAF)	High
3	External	Session	Spoofing	Session	Confidentialia	Secure	High

	Hackers	hijacking		Tokens	lity	cookies (HttpOnly, Secure), TLS encryption	
4	Insider Students	Grade manipulation attempt	Tampering	Academic Records	Integrity	Role-Based Access Control (RBAC), Logging, Approval workflows	Low – Medium
5	Insider Students	Accessing exam bank early	Information Disclosure	Exam Content	Confidentiality	Strict access control, Encryption at rest	Medium
6	Malicious Administrator	Unauthorized grade changes	Tampering	Academic Records	Integrity	Segregation of duties, Continuous audit logs, Dual approval system	Low (technical) / High (impact)
7	Malicious Administrator	Log deletion	Repudiation	Audit Logs	Integrity & Accountability	Immutable logging, Log backups, SIEM monitoring	Medium
8	Organized Cybercriminal Groups	Ransomware attack	Denial of Service	Database Servers, Cloud	Availability	Regular encrypted backups,	Very High

				Infrastructure		Endpoint detection, Network segmentation	
9	Organized Cybercriminal Groups	Large-scale data breach	Information Disclosure	Student Personal Data	Confidentiality	Encryption, Continuous monitoring, Data Loss Prevention (DLP)	Very High
10	External Hackers	Privilege escalation exploit	Elevation of Privilege	User Accounts	Confidentiality & Integrity	Least privilege principle, Access reviews, Patch management	High
11	External Hackers	Distributed Denial-of-Service (DDoS)	Denial of Service	Web Application Servers	Availability	DDoS protection services, Load balancing, Rate limiting	High
12	Insider Students	Assignment timestamp manipulation	Repudiation / Tampering	Academic Records	Integrity	Secure timestamping, Digital signatures, Logging	Low – Medium

3. Software Requirements

SQUARE-Lite process applied.

3.1 Agree on Definitions

Before defining security requirements, the team agreed on a common understanding of key security concepts to ensure consistency throughout the risk analysis and requirement elicitation process. Establishing shared definitions helps eliminate ambiguity and ensures that all stakeholders interpret security goals uniformly.

Term	Expanded Definition
Confidentiality	The protection of sensitive information from unauthorized access or disclosure. In Canvas LMS, this includes protecting student personal data, grades, authentication credentials, and exam content from external attackers and unauthorized insiders.
Integrity	The assurance that data remains accurate, complete, and unaltered except by authorized users. For the LMS, integrity ensures that grades, submissions, and audit logs cannot be modified maliciously or accidentally.
Availability	Ensuring that the system and its services remain accessible and operational for authorized users whenever needed. Since Canvas supports academic

	activities, high availability (e.g., 99.9% uptime) is essential.
Authentication	The process of verifying the identity of a user or system before granting access. This may include passwords, Multi-Factor Authentication (MFA), and secure tokens.
Authorization	The process of granting users appropriate permissions after authentication, based on their role (Student, Instructor, Administrator).
Least Privilege	A security principle that ensures users are given only the minimum access rights necessary to perform their tasks, reducing the risk of misuse or privilege escalation.
Non-repudiation	The ability to prove that a specific action was performed by a particular user, preventing them from denying their actions. This is achieved through secure logging, timestamps, and audit trails.
Threat	Any circumstance or event that has the potential to exploit a vulnerability and cause harm to system assets.
Vulnerability	A weakness in the system design, implementation, configuration, or process that can be exploited by a threat actor.
Risk	The combination of the likelihood of a threat exploiting a vulnerability and

	the resulting impact on system assets.
--	--

3.2 Identify Security Goals

The main goals include:

- i. Protect academic data confidentiality.
- ii. Ensure grade integrity.
- iii. Prevent unauthorized system access.
- iv. Maintain 99.9% availability.
- v. Provide traceability of actions.
- vi. Ensure compliance with FERPA-like regulations.

3.3 Elicit Security Requirements

Each of the security requirements for Canvas LMS is designed based on the SMART criteria, i.e., Specific, Measurable, Achievable, Relevant, and Time-bound. The distinctiveness of each requirement is that control should be applied in a particular manner, the success of which can be evaluated, and time limits set for when the requirement should be fulfilled (Cloud Security Alliance, 2023). Also, the requirements are checked against the CIA triad and explained using the STRIDE threat model to demonstrate that they directly address identified risks. All of the requirements address at least one of the STRIDE threats. To illustrate:

- SR-H1 (Multi-Factor Authentication) addresses the Spoofing and Elevation of Privilege risks by enhancing authentication and decreasing the chance of compromised accounts (European Union Agency for Cybersecurity, 2023).

- SR-H7 (Input Validation and Sanitization) helps address Tampering and Information Disclosure by preventing injection attacks and unauthorized data manipulation.
- SR-H19 (DDoS Protection) directly counters Denial-of-Service attacks by maintaining system availability during high traffic or malicious attacks. This systematic congruency ensures that all security requirements are not only clearly defined but also strategically overlaid on real-world threat conditions.

3.4 Prioritization of requirements

Requirements were prioritized based on:

- Risk severity
- Asset criticality
- Regulatory compliance needs
- Likelihood of attack
- Implementation feasibility

4. Team Contribution

Section	Author	Reviewer
---------	--------	----------

Product Assets	Haifa Alangari	Joud Alnitaifi
Threat Modeling	Haya Albakr	Nora Alomier
Security Requirements	All	Cross-reviewed
Final Review	Nora Alomier	Haifa Alangari

References

- Cloud Security Alliance. (2023). *Security guidance for critical areas of focus in cloud computing (Version 4.1)*. <https://cloudsecurityalliance.org>
- EDUCAUSE. (2023). *Top IT issues 2023: Resilience and adaptability in higher education*. <https://www.educause.edu>
- European Union Agency for Cybersecurity. (2023). *ENISA threat landscape 2023*. <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2023>
- International Organization for Standardization. (2022). *ISO/IEC 27001:2022 information security management systems — Requirements*. ISO.
- National Institute of Standards and Technology. (2022). *Secure software development framework (SSDF) version 1.1 (SP 800-218)*. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-218>

National Institute of Standards and Technology. (2023). *Security and privacy controls for information systems and organizations (SP 800-53 Rev. 5, Update 1)*. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-53r5>

OWASP Foundation. (2023). *OWASP top 10: The ten most critical web application security risks*. <https://owasp.org/www-project-top-ten/>

References

Instructure. (n.d.). *Canvas LMS security overview*. <https://www.instructure.com/security>

Instructure. (n.d.). Canvas API documentation. <https://canvas.instructure.com/doc/api/>

OWASP Foundation. (2021). OWASP Top 10: The ten most critical web application security risks. <https://owasp.org/www-project-top-ten/>

National Institute of Standards and Technology. (2020). Security and privacy controls for information systems and organizations (NIST SP 800-53 Rev. 5).
<https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>

EDUCAUSE. (n.d.). Cybersecurity in higher education. <https://www.educause.edu/focus-areas-and-initiatives/cybersecurity>

U.S. Department of Education. (n.d.). Family Educational Rights and Privacy Act (FERPA).
<https://www.ed.gov/laws-and-policy/privacy-and-civil-liberties/ferpa>

Prince Sultan University

College of Computer and Info Sciences / Department of Computer Science

Term 252

CYS402 : Secure Software Development

Section : 1407

Canvas (LMS)

Prepared by:

Joud AlNitaifi - 221410121

Haifa Alangari - 221410567

Nora Alomier - 222410172

Haya Albakr - 219410618

Instructor:

Dr. Zainab Masood

System Architecture Overview	3
Subsystems and Their Roles	3
User Interface (UI) System	4
Application Logic Subsystem	4
Authentication and Access Control System	4
Database Management Subsystem	5
API Integration System	5
The Logging and Monitoring Subsystem	6
Security Specific Subsystems	6
Authentication Security	6
Encryption Services	6
Access Control System	6
Audit Logging System	7
The Cost of Developer Errors	7
Most Vulnerable Code level Subsystems	7

Security is integrated into the system architecture	8
Risks from External Interactions	8
Threat Modeling	8
Machine Boundaries	8
Trust Boundaries	9
Threat Model with Threats and Mitigation Methods	9
Resources for Threat Model Tracking and Evaluation	10
Assets to Subsystems Mapping	11
Class Diagram	11
Secure Design Patterns and Principles	20
Secure Design Patterns	20
Impact of Compromising One Subsystem on the Whole System	20
Ways in Which Architecture Could Change Over Time	21
Team contribution Table	22
References	22

System Architecture Overview

Subsystems and Their Roles

The Canvas Learning Management System (LMS) is designed using multiple interconnected subsystems that work together to support e-learning activities while maintaining security, reliability, and scalability. Each subsystem performs a specific function within the system architecture.

Architecture Might Evolve Over Time

The design is intended to support future modifications and development over time. As the system expands, additional features like automated grading tools, analytics powered by machine learning for assessing student performance, or connections with other educational platforms could be incorporated. Every new feature requires thorough examination for potential security risks, with appropriate controls established to minimize them. With the growth in user numbers, the system might require horizontal scaling, which may entail incorporating additional web servers or shifting to a microservices architecture. The design must guarantee that scalability does not interfere with security measures. Moreover, with the rise of new security threats like quantum computing possibly compromising encryption, the security architecture of the system needs to adapt. For example, enhancing encryption standards to include quantum-secure algorithms could become essential. Finally, the system must stay adaptable enough to incorporate new technologies such as blockchain for secure documentation or AI for smart tutoring, guaranteeing its adjustment to continuous advancements in educational technology.

User Interface (UI) System

The User Interface (UI) system provides the primary point of interaction between users and the Canvas platform. Accessible via web browsers and mobile applications, it allows students, instructors, and administrators to interact with the system.

Through this subsystem, users can log in to the platform, access course materials, submit assignments, participate in discussions, view grades, and manage course settings. The UI system communicates directly with the application logic layer, ensuring that all user actions are correctly transmitted to backend services.

Although the UI system does not typically store sensitive data directly, it plays a crucial role in validating user input and preventing malicious input that could lead to security vulnerabilities such as cross-site scripting (XSS) attacks.

Application Logic Subsystem

The Application Logic Subsystem handles all functional operations within the system. It manages course operations, handles requests from the user interface, and coordinates communication between different system components.

This subsystem ensures the application of rules within the Learning Management System (LMS) environment. For example, it verifies student registration in a course before granting access to its content and ensures that grade modification is restricted to instructors only.

Because the application layer handles most user-generated requests, it must be carefully designed to prevent vulnerabilities such as injection attacks, access control failures, or session mismanagement.

Authentication and Access Control System

The authentication system is responsible for verifying user identities before granting them access to the system. Users must authenticate using their login credentials before accessing system services.

After authentication is complete, the authorization component ensures that each user only has access to resources that correspond to their defined role. The Canvas Learning Management System typically uses a Role-Based Access Control (RBAC) system to manage the permissions of students, instructors, and administrators.

For example, students can submit assignments and view grades, while instructors can manage course content and evaluate student performance. Administrators may have broader permissions related to system configuration and user management.

This subsystem is one of the most critical components of the system from a security standpoint, as any vulnerabilities in the authentication mechanisms could allow unauthorized users to access sensitive academic data.

Database Management Subsystem

The database management subsystem stores and manages all permanent system data. This includes students' personal information, course content, academic records, grades, authentication credentials, and system logs.

Because the database stores highly sensitive information, it is one of the most critical assets of the system architecture. Unauthorized access to the database can have serious consequences, such as data breaches, manipulation of academic records, or disclosure of confidential information.

Therefore, the database management subsystem must implement strong access controls, encryption mechanisms, and secure communication channels to ensure data confidentiality and integrity.

API Integration System

The Canvas Learning Management System integrates with various external services, such as third-party learning tools, learning platforms, analytics systems, and mobile applications. An API integration system enables secure communication between Canvas and these external systems.

API tokens and integration keys are used to authenticate communication between the systems without requiring manual login each time. However, if these tokens are compromised or stored incorrectly, attackers could gain unauthorized access to system resources.

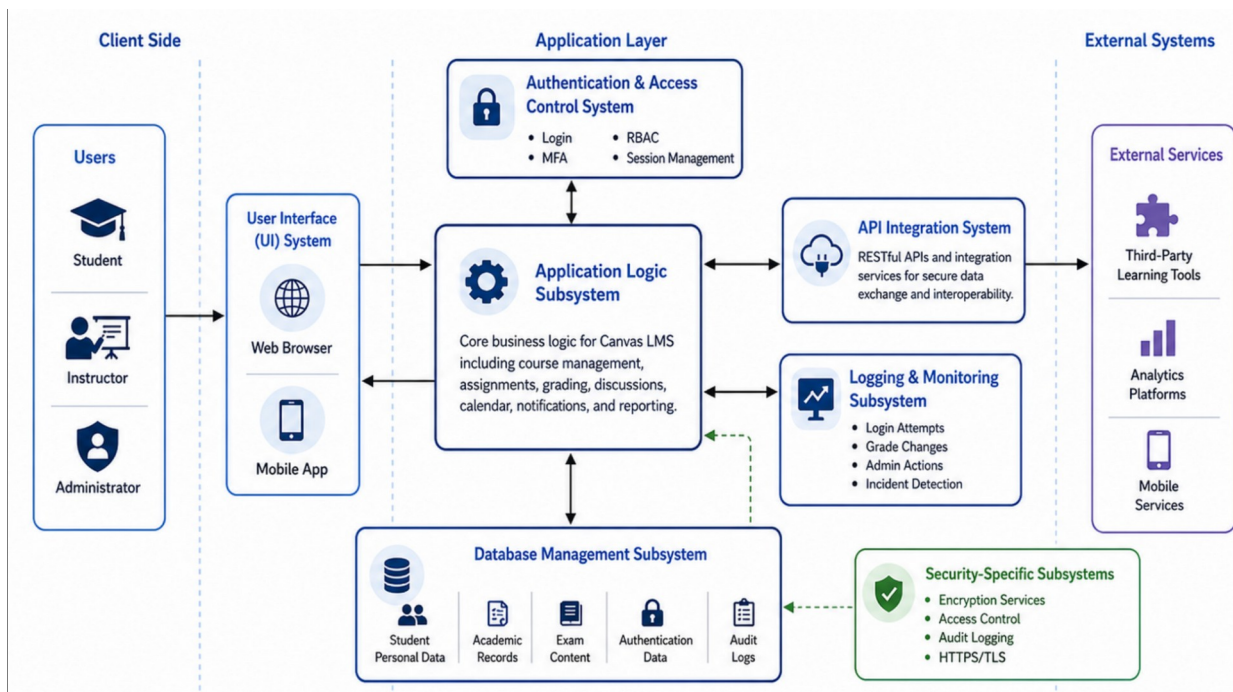
Therefore, the API integration system must implement robust authentication mechanisms, strict access policies, and encryption for data exchanged between the external services.

The Logging and Monitoring Subsystem

The logging and monitoring subsystem logs system activities and security-related events. Examples include login attempts, grade changes, course material modifications, and administrative actions.

These audit logs are essential for security monitoring, incident detection, and digital forensics. In the event of suspicious activity, administrators can review these logs to determine what happened and which users were involved.

Proper protection of logging systems is also crucial, as attackers may attempt to modify or delete logs to conceal their malicious activities.



Security Specific Subsystems

Several subsystems within the Canvas Learning Management System are specifically designed to enhance system security and protect sensitive data.

Authentication Security

The authentication system ensures that only authorized users can access the Learning Management System. Password-based authentication, along with multi-factor authentication, helps reduce the risk of unauthorized account access.

Encryption Services

Encryption mechanisms protect sensitive information during storage and transmission. Canvas typically uses HTTPS and TLS protocols to encrypt communications between users and the system.

Access Control System

The access control subsystem enforces permissions using role-based access control policies. This ensures that users cannot access or modify information beyond their authorized permissions.

Audit Logging System

The audit logging subsystem ensures that system activities can be tracked. It logs significant events, such as login attempts, configuration changes, and grade modifications, to support accountability and incident investigation.

The Cost of Developer Errors

Errors made by developers in various subsystems can lead to a variety of security risks within the system.

Errors in the user interface system can allow attackers to inject malware into the application if input validation is not implemented correctly. These vulnerabilities can lead to cross-site scripting attacks.

Errors in the application logic system can allow users to bypass security controls or manipulate system functionality. For example, incorrect access checks could allow students to access resources intended for instructors only.

Errors in the authentication system can allow attackers to bypass login mechanisms, resulting in unauthorized access to student accounts or administrator privileges.

Incorrect configurations in the database system can expose sensitive information or allow unauthorized modifications to academic records.

Because these errors can significantly impact system security, secure programming practices and security testing should be implemented throughout the software development lifecycle.

Most Vulnerable Code level Subsystems

Certain subsystems are more vulnerable to code-level vulnerabilities due to their direct interaction with users or external systems.

The web application layer is particularly vulnerable because it handles large amounts of user input. Improper input validation can allow for injection attacks, such as SQL injection or cross-site scripting (XSS).

The API integration system is also a high-risk component because it communicates with external systems. Weak API authentication mechanisms or exposed tokens can allow attackers unauthorized access to system resources.

Similarly, the authentication system is a prime target for attacks because compromising user credentials can grant attackers direct access to sensitive system data.

Security is integrated into the system architecture

Security is integrated into the architecture of the Canvas Learning Management System (LMS) using several design principles. The system follows a multi-layered defense principle, protected by multiple layers of security controls. These controls include authentication mechanisms, encryption, access control policies, and monitoring systems.

The architecture also follows the principle of least privilege, ensuring that users have the minimum privileges necessary to perform their tasks.

In addition, the system enforces secure communication protocols such as HTTPS and TLS to protect data transmitted between users and the system.

Finally, centralized logging and monitoring mechanisms help detect suspicious activity and enable rapid response to potential security incidents.

Risks from External Interactions

Canvas LMS interacts with several external services including third-party learning tools, analytics platforms, mobile applications, and educational integrations.

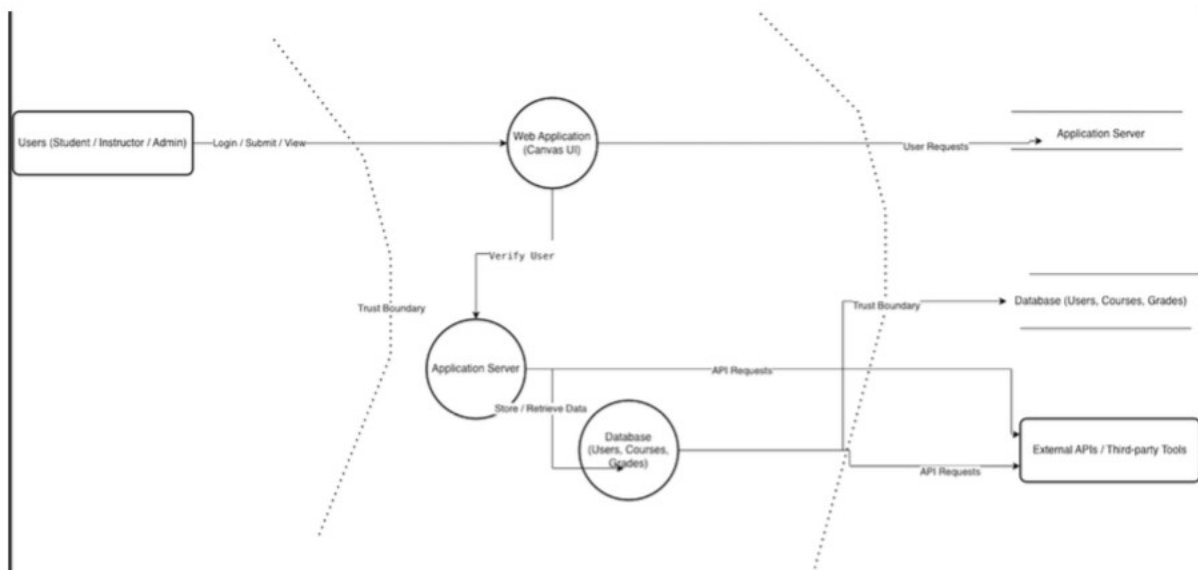
These external interactions introduce potential security risks. For example, exposed API tokens may allow attackers to access system resources without authentication.

Third-party integrations may also introduce supply-chain vulnerabilities if external components contain security flaws.

Therefore, strict authentication, secure API management, and continuous monitoring must be implemented to protect the system from risks introduced by external interactions.

Threat Modeling

Machine Boundaries



Machine defines the boundaries of separation between different computing environments within the system architecture.

In Canvas LMS, the primary machine boundary exists between user devices and the Canvas application servers. Students and instructors access the system through web browsers or mobile applications, which communicate with the web servers over the internet.

Another machine boundary exists between the web application servers and the database servers that store academic records and other sensitive information.

External integrations such as third-party learning tools and cloud services may introduce additional machine boundaries, expanding the system's architecture.

Trust Boundaries

Trust boundaries represent points within the system where different levels of trust exist.

One important trust boundary exists between users and the web application interface. User inputs should never be fully trusted and must always be validated before processing.

Another trust boundary exists between the web application layer and the database subsystem. Strict authorization checks must be enforced to ensure that only legitimate queries are executed. A third trust boundary exists between Canvas LMS and external services accessed through APIs. Since external systems may not follow the same security standards, additional authentication and validation mechanisms are required.

Threat Model with Threats and Mitigation Methods

The following is a more detailed description of the threat model with a clear list of threats and how to mitigate them:

Threat 1: Cross-Site Scripting (XSS)

Threat Description: Malicious users may inject malicious scripts into the application, which can then be executed by other users.

Solution: Implement validation for all user input to ensure that potentially harmful content is cleaned up before being processed or displayed on the webpage.

Threat 2: SQL Injection

Threat Description: Attackers may attempt to manipulate SQL queries to access, modify, or delete sensitive data in the database.

Solution: Use flagged queries and prepared statements at the database layer to prevent SQL injection attacks.

Threat 3: Authentication Bypass

Threat Description: Attackers may attempt to bypass the authentication process to gain unauthorized access to user accounts.

Preventive Measures: Implement multi-factor authentication (MFA) and ensure that passwords are securely encrypted using bcrypt or Argon2. Additionally, implement robust session management and automatic session expiration. Threat 4: API Token Exposure

Threat Description: Exposed API tokens may allow unauthorized external services to access system resources.

Preventive Measures: Store API tokens securely and use environment variables for storage. Implement token expiration and revocation mechanisms, and strictly limit the API request rate.

Threat 5: Privilege Escalation

Threat Description: Users may attempt to access resources or functions they are not authorized to access.

Preventive Measures: Use Role-Based Access Control (RBAC) and apply the principle of least privilege throughout the system.

Resources for Threat Model Tracking and Evaluation

The Impact of Location on Exploit Likelihood:

Database Subsystem (Student Personal Data, Academic Records): The assets within the database subsystem are extremely sensitive. Any weaknesses in the database management system (e.g., misconfiguration or SQL injection) raise the chances of exploitation. These must be protected with robust encryption and access control measures.

Are There Any Assets Unaccounted For?

Missing Assets: One potential missing asset might be transaction records for API interactions. These logs are crucial for identifying and examining any harmful API usage.

Areas with Excessive Assets:

Authentication Subsystem: The asset of Authentication Credentials is an essential component of the system. Given that credentials are essential to user identity and access, this asset resides in the authentication subsystem, potentially making it a target. It's essential to guarantee that robust encryption and multi-factor authentication (MFA) are implemented

Assets to Subsystems Mapping

Asset	Subsystem
Student Personal Data	Database Subsystem
Academic Records (Grades, Submissions)	Database Subsystem
Authentication Credentials	Authentication Subsystem
Exam Content	Database Subsystem
API Tokens	API Integration Subsystem
Audit Logs	Logging and Monitoring Subsystem

Class Diagram

The detailed design of the Canvas Learning Management System (LMS) focuses on the core functional components responsible for user management, course management, assessment handling, authentication, and security monitoring. The class diagram illustrates the relationships

between the main system entities and demonstrates how responsibilities are distributed across system components to ensure maintainability and security.

Main Classes in the System

The system consists of several key classes that represent the fundamental components of the LMS architecture.

1. User Class

The User class represents all system users including students, instructors, and administrators. It serves as the parent class for role-specific subclasses.

Attributes

- userID
- name
- email
- passwordHash
- role

Methods

- login()

- logout()
- updateProfile()

This class is responsible for managing user identity and authentication interactions.

2. Student Class

The Student class extends the User class and represents students enrolled in courses.

Attributes

- studentID
- enrolledCourses

Methods

- submitAssignment()
- viewGrades()
- accessCourseMaterials()

Students interact primarily with course content and assessments while having limited permissions according to role-based access control policies.

3. Instructor Class

The Instructor class also extends the User class and represents course instructors.

Attributes

- instructorID
- assignedCourses

Methods

- createAssignment()
- gradeSubmission()
- uploadCourseMaterial()

Instructors have elevated privileges compared to students but are restricted from administrative functions.

4. Administrator Class

The Administrator class represents system administrators responsible for system management.

Attributes

- adminID
- privileges

Methods

- manageUsers()
- configureSystem()
- viewAuditLogs()

Administrators manage platform configurations, user roles, and system-level settings.

5. Course Class

The Course class represents academic courses within the LMS.

Attributes

- courseID
- courseTitle
- instructorID
- enrolledStudents

Methods

- addStudent()
- removeStudent()
- publishContent()

Courses serve as containers for assignments, materials, and assessments.

6. Assignment Class

The Assignment class represents coursework given to students.

Attributes

- assignmentID

- title
- description
- deadline

Methods

- submitAssignment()
- evaluateSubmission()

Assignments store submission data and grading results.

7. Submission Class

The Submission class stores the work submitted by students.

Attributes

- submissionID
- studentID
- assignmentID
- timestamp
- grade

Methods

- uploadFile()

- updateSubmission()
- recordGrade()

Submissions maintain academic records and ensure submission integrity.

8. AuthenticationService Class

The AuthenticationService class manages secure authentication and access control.

Attributes

- token
- sessionID

Methods

- authenticateUser()
- verifyToken()
- generateSession()

This class is responsible for validating login credentials and issuing secure session tokens.

9. AuditLog Class

The AuditLog class records system activities for security monitoring.

Attributes

- logID

- userID
- action
- timestamp

Methods

- recordEvent()
- retrieveLogs()

Audit logs support accountability, forensic investigation, and compliance monitoring.

Class Relationships

The system implements the following relationships:

Inheritance

- User $\rightarrow$ Student
- User $\rightarrow$ Instructor
- User $\rightarrow$ Administrator

Associations

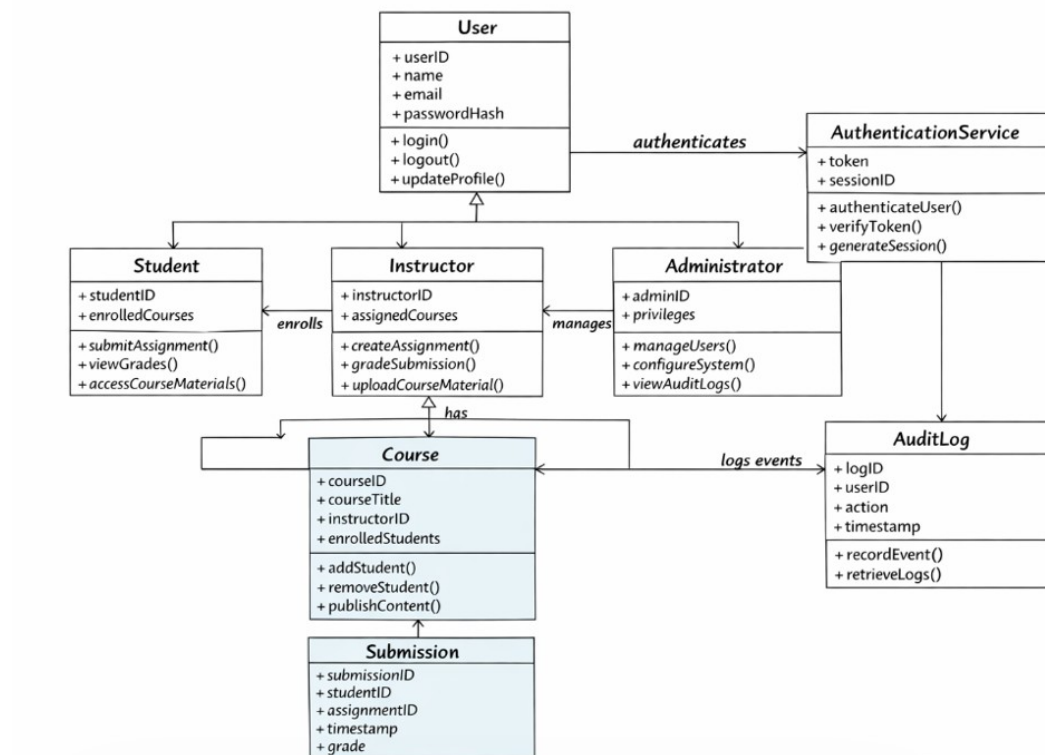
- Instructor manages Course
- Student enrolls in Course

- Course contains Assignment
- Assignment receives Submission

Dependency

- AuthenticationService verifies User
- AuditLog records actions performed by User

This architecture ensures modularity and separation of responsibilities across system components.



4. Secure Design Patterns and Principles

To strengthen system security, several secure design patterns and principles were incorporated into the LMS architecture.

1. Role-Based Access Control (RBAC)

Role-Based Access Control is implemented to regulate user permissions based on assigned roles.

In the system, three primary roles exist:

- Student
- Instructor
- Administrator

Every position is endowed with the rights that it needs to carry out its duties. To illustrate, students are capable of submitting assignments and seeing grades but cannot change course content or system settings. RBAC implements the least privilege principle, whereby users cannot access resources that are not within their permission limits. This practice would greatly minimize the chances of privilege escalation and unauthorized modifications of data.

2. Least Privilege Principle

The system applies the least privilege principle, which ensures that users and processes are given only the minimum permissions required to perform their tasks.

For example:

- Students cannot access grading functions.
- Instructors cannot modify system-level configurations.
- Administrative privileges are restricted to authorized administrators.

By minimizing permissions, the system limits potential damage caused by compromised accounts or insider threats.

3. Secure Authentication pattern

A secure authentication mechanism is implemented through the AuthenticationService class.

Security features include:

- Password hashing
- Multi-Factor Authentication (MFA)
- Secure session tokens
- Session timeout management

These controls protect against common threats such as:

- credential theft
- brute-force attacks
- session hijacking

Strong authentication ensures that only verified users gain access to the system.

4. Input Validation and Output encoding

The system applies strict input validation mechanisms to prevent injection-based attacks.

All user inputs are validated before being processed by the system. This prevents vulnerabilities such as:

- SQL injection
- Cross-Site Scripting (XSS)
- command injection

Output encoding is also applied when displaying user-generated content to prevent malicious scripts from executing in user browsers.

5. Secure Logging and Audit Trail

The AuditLog component provides comprehensive logging of security-sensitive actions such as:

- login attempts
- grade modifications
- role changes
- administrative activities

These logs support non-repudiation, meaning users cannot deny actions performed using their accounts. Logging also enables rapid detection of suspicious activities and assists in incident response and forensic analysis.

6. Defense in Depth

The system follows a defense-in-depth strategy, where multiple layers of security controls protect the system.

Examples include:

- authentication controls
- access control mechanisms
- encryption of sensitive data
- secure session management
- monitoring through audit logs

If one security layer fails, additional controls remain in place to prevent attackers from compromising the system.

Secure Design Patterns and Principles

Secure Design Patterns

Pattern	Where Applied	Security Benefit
Role-Based Access Control	Access Control System	Prevents unauthorized access to system resources

Secure Key Storage	Authentication Subsystem	Protects sensitive credentials
Secure Communication (TLS)	Web Communication	Ensures confidentiality of transmitted data
Centralized Logging	Logging Subsystem	Enables monitoring and incident investigation

Impact of Compromising One Subsystem on the Whole System

Compromising specific subsystems might lead to significant effects throughout the entire system. Here's a way that a failure in one sector might impact others:

Authentication Subsystem: In the event of a breach in the authentication subsystem, attackers might obtain unauthorized access to every area of the system, circumventing role-based limitations and retrieving confidential information such as student records or grades.

Database Management Subsystem: Should the database be breached, attackers might modify academic records, change grades, or potentially erase essential data. This would interrupt the complete operation of the system and might result in serious legal and reputational repercussions.

API Integration System: Should an external integration be breached, malicious individuals might exploit this access to alter data across systems, damaging course materials or student records.

Overall, defense in depth has been applied across the system to guarantee that the compromise of a single subsystem does not result in the total compromise of the entire system.

Ways in Which Architecture Could Change Over Time

The structure is intended to support future modifications and developments over time. This is a potential evolution of the system:

Incorporating Additional Features: As the system expands, features like automated grading tools, analytics driven by machine learning for student performance, or connections with other educational platforms can be added. Every new feature must be thoroughly examined for security vulnerabilities, and appropriate measures should be implemented.

Scalability: With an increase in users, the system may require horizontal scaling. This might mean incorporating additional web servers or adopting a microservices architecture. The system must be created to expand with little disturbance to security protocols.

Security Progression: With the rise of new security threats (e.g., quantum computing compromising encryption), the security architecture of the system must adapt. For instance, enhancing encryption methods to incorporate quantum-resistant algorithms may be necessary in the future. **Integration of New Technologies:** New technologies such as blockchain for secure records or AI for intelligent tutoring could be incorporated. The architecture should remain flexible and adaptable to new innovations in educational technology.

To address security patterns according to the OpenClinica concept, it is essential to implement best practices in secure design and apply the security patterns outlined by global standards such

as OWASP and SANS. One of the key security patterns is the Least Privilege Principle, which ensures that each user and component in the system only has the minimum permissions required to perform their specific tasks. In OpenClinica, this means limiting access to sensitive medical data to authorized personnel only, such as doctors or researchers, while administrative staff might have access to other functionalities but not to medical records. Another important pattern is Multi-factor Authentication (MFA), which enhances security by requiring users to verify their identity through more than one method, such as a password combined with a code sent via email or an authentication app. Encryption in Transit is also crucial; it ensures that all data exchanged between systems, particularly sensitive information like medical records, is securely encrypted during transmission, preventing interception by unauthorized parties. Additionally, Role-Based Access Control (RBAC) helps by granting users permissions based on their roles, ensuring that doctors, nurses, and administrators only access data relevant to their responsibilities.

Implementing Audit Logging is another important security measure, ensuring that all significant actions within the system, such as accessing or modifying medical data, are recorded for accountability and future review. Data Segmentation further strengthens security by dividing data into categories and applying different levels of access control to each, ensuring that sensitive data, such as patient medical records, is treated with higher security measures. To ensure the integrity of this data, Data Integrity measures like hashing and digital signatures should be used, ensuring that data cannot be altered without proper authorization. To protect the system from common vulnerabilities, Protection Against Known Attacks such as SQL Injection and Cross-Site Scripting (XSS) should be implemented through input filtering and validation. An effective Incident Response plan should also be in place, ensuring that any security breaches or attacks can be rapidly addressed, minimizing damage. Finally, DDoS Protection should be

applied, ensuring the system remains operational even under high traffic conditions or malicious attack attempts. Integrating these security patterns into the design of OpenClinica ensures that the system is resilient to attacks and meets the stringent requirements for safeguarding sensitive medical data.

Team contribution Table

Part of the Project	Team Member Responsible	Team Member Reviewing
Architecture Overview	Joud AlNitaifi	Haifa Alangari
Threat Model	Haifa Alangari	Nora Alomier
Assets to Threat Mapping	Nora Alomier	Haya Albakr
Class Diagrams	Haya Albakr	Joud AlNitaifi

Secure Design Patterns	Joud AlNitaifi	Haifa Alangari
Risk and Evolution of System	Joud AlNitaifi	Nora Alomier

References

El-Afoui, K. (2024).

Security challenges and protection mechanisms in e-learning systems. International Scientific Journal of Engineering and Management (ISJEM).

https://isjem.com/wp-content/uploads/2024/04/EL-AFOUI-KAMAL_-2024.pdf

Aydin, C. H., & Tasci, D. (2012).

Security issues in e-learning systems. In Advances in Information Systems Development (pp. 111–124). Springer.

https://doi.org/10.1007/978-3-642-31753-8_9

IEEE. (2012).

Security framework for e-learning systems. IEEE Conference Publication.

<https://ieeexplore.ieee.org/document/6320525>

SSRN. (2024).

Cybersecurity challenges in modern learning management systems.

https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5259056

Prince Sultan University

College of Computer and Info Sciences / Department of Computer Science

Term 252

CYS402 : Secure Software Development

Section : 1407

Canvas (LMS)

Prepared by:

Joud AlNitaifi - 221410121

Haifa Alangari - 221410567

Nora Alomier - 222410172

Haya Albakr - 219410618

Instructor:

Dr. Zainab Masood

Executive Summary

The report satisfies the Phase 4: Secure Implementation criteria by providing a detailed code-level security analysis of the official Canvas LMS open-source codebase. All team members conducted a manual security code review of a single critical function or method, applying the OWASP Top 10 (2021), the OWASP Code Review Guide, and the OWASP Application Security Verification Standard (ASVS) as the main evaluation patterns. Moreover, every member used SonarCloud to perform automated static analysis on the same repository and chose one of the security hotspots or problems identified. The complete continuity and alignment of all findings across the entire project lifecycle are explicitly mapped to assets, subsystems, threats, and architectural components documented in Phases 1-3.

Individual Member Responsibilities

Each team member independently selected and analyzed one security-related function from the Canvas LMS GitHub repository. Every member conducted:

- Manual secure code review using OWASP Top 10 and OWASP ASVS
- Independent SonarCloud static analysis
- Vulnerability assessment and mitigation recommendations
- Secure implementation improvement review

All analyses were completed individually and mapped to the security architecture identified in previous project phases.

Manual Code Review – Individual Analysis

1. **Function:** `authenticate_one_time_password` (app/models/user.rb)

Code location: app/models/user.rb (lines 3911, main branch)

What the function does:

This method verifies a user-submitted one-time password (OTP) during multi-factor authentication (MFA). It performs an atomic lookup for an unused OTP record, marks it as used, and returns the record only if exactly one row was successfully updated (preventing race conditions).

```
def authenticate_one_time_password(code)

  result = one_time_passwords.find_by(code: code, used: false)

  return unless result

  return unless one_time_passwords.where(used: false, id: result.id)

    .update_all(used: true, updated_at: Time.now.utc) == 1

  result

end
```

```

3908     end
3909   end
3910 end
3911 end
3912
3913 def authenticate_one_time_password(code)
3914   result = one_time_passwords.find_by(code:, used: false)
3915   return unless result
3916   # atomically update used
3917   return unless one_time_passwords.where(used: false, id: result).update_all(used: true, updated_at: Time.now.utc) == 1
3918
3919   result
3920 end
3921
3922 def generate_one_time_passwords(regenerate: false)
3923   regenerate ||= !one_time_passwords.exists?
3924   return unless regenerate
3925 end

```

Github link: <https://github.com/instructure/canvas-lms/tree/master/app>

Why this function was chosen (Security relevance):

MFA is the primary defense against credential-stuffing and phishing attacks. This function is the single point of truth for OTP validation in the entire authentication flow. Any flaw here directly undermines the entire authentication subsystem. Assets / Components affected:

- Authentication Credentials (MFA secrets and OTP records) – Phase 2, Data Assets #4
- Student/Instructor accounts and academic records – Phase 2, Data Assets #1 & #2

Subsystem / Module: Authentication Security Subsystem (Phase 3, Section “Security Specific Subsystems”).

Security risks/concerns identified:

- Relies on database-level atomicity; any change in the ActiveRecord query could reintroduce race conditions.
- No explicit constant-time comparison for the OTP code (timing side-channel risk).

- Rate limiting is handled in the controller, not inside this model method.

Potential issues if errors are introduced:

- Successful brute-force or replay attack → full account takeover → attacker can view/change grades, submit assignments as any student, or access private course materials (direct violation of CIA triad: Confidentiality & Integrity).
- Academic integrity collapse and FERPA violations (Phase 1 consequences).

Vulnerabilities / Bad coding practices / Design issues (OWASP):

- OWASP Top 10 A07: Identification and Authentication Failures – Partial compliance with ASVS 2.5.1 (MFA verification).
- Minor design issue: missing constant-time comparison (recommend SecureCompare gem).
- Overall assessment: Low risk in current implementation, but requires hardening for production educational environments.

2. **Function:** session_token (app/controllers/login_controller.rb)

Code location: app/controllers/login_controller.rb (around line 180)

What the function does:

Generates a signed session token after successful login, validates the optional return_to URL, and redirects the user while appending the token.

Why chosen:

Session token generation and return-to URL validation are classic vectors for session fixation, open redirects, and token leakage — two of the most common attacks on learning platforms.

Assets Affected:

Session Tokens & Authentication Credentials (Phase 2).

Subsystem: Authentication and Access Control System (Phase 3).

Security risks/concerns:

- The return_to parameter is sanitized but still depends on the correct server configuration of trusted hosts.
- Token appears in query string (visible in browser history, logs, and referer headers).

Potential issues if errors are introduced: Open redirect → phishing campaign targeting students → credential theft → mass data breach (Phase 1 risk #2: Unauthorized Access).

OWASP analysis:

- OWASP Top 10 A01: Broken Access Control & A05: Security Misconfiguration.
- ASVS 3.1.1 (URL validation) is satisfied but could be strengthened with an explicit allow-list.
- Recommendation: move token to an HTTP-only cookie instead of a URL.

```
login_controller.rb X
C:\Users\USER> Downloads > canvas-lms-master > canvas-lms-master > app > controllers > login_controller.rb
21 class LoginController < ApplicationController
101
102 # GET /login/session_token
103 def session_token
104 # must be used from API
105 return render_unauthorized_action unless @access_token
106
107 # verify that we're sending them back to a host from the same instance
108 begin
109 | return_to = URI.parse(params[:return_to] || request.referrer || root_url)
110 rescue URI::InvalidURIError => e
111 | Canvas::Errors.capture_exception(:login, e, :info)
112 | return render json: { error: I18n.t("Invalid redirect URL") }, status: :bad_request
113 end
114 return render_unauthorized_action unless return_to.absolute?
115 return render_unauthorized_action unless return_to.scheme == request.scheme
116
117 host = return_to.host
118 return render_unauthorized_action unless host.casecmp?(request.host)
119
120 consent_from_mobile = if params[:mobile_consent].present?
121 | params[:mobile_consent] == "true"
122 | else
123 | nil
124 | end
125
126 login_pseudonym = @real_current_pseudonym || @current_pseudonym
127 token = SessionToken.new(login_pseudonym.global_id,
128 | current_user_id: @real_current_user ? @current_user.global_id : nil,
129 | used_remember_me_token: true,
```

Github link: <https://github.com/instructure/canvas-lms/tree/master/app/controllers>

3. **Function:** otp_secret_key= (setter in app/models/user.rb)

Code Location: app/models/user.rb (OTP secret setter method)

What the function does:

Encrypts the OTP secret key using Canvas::Security::encrypt_password before persisting it to the database.

Why chosen:

This is the **only place** where MFA secrets are encrypted at rest. It directly protects the confidentiality of the strongest authentication factor.

Assets affected:

- Authentication Credentials (MFA secrets) – Phase 2.

Subsystem: Encryption Services + Authentication Security Subsystem (Phase 3).

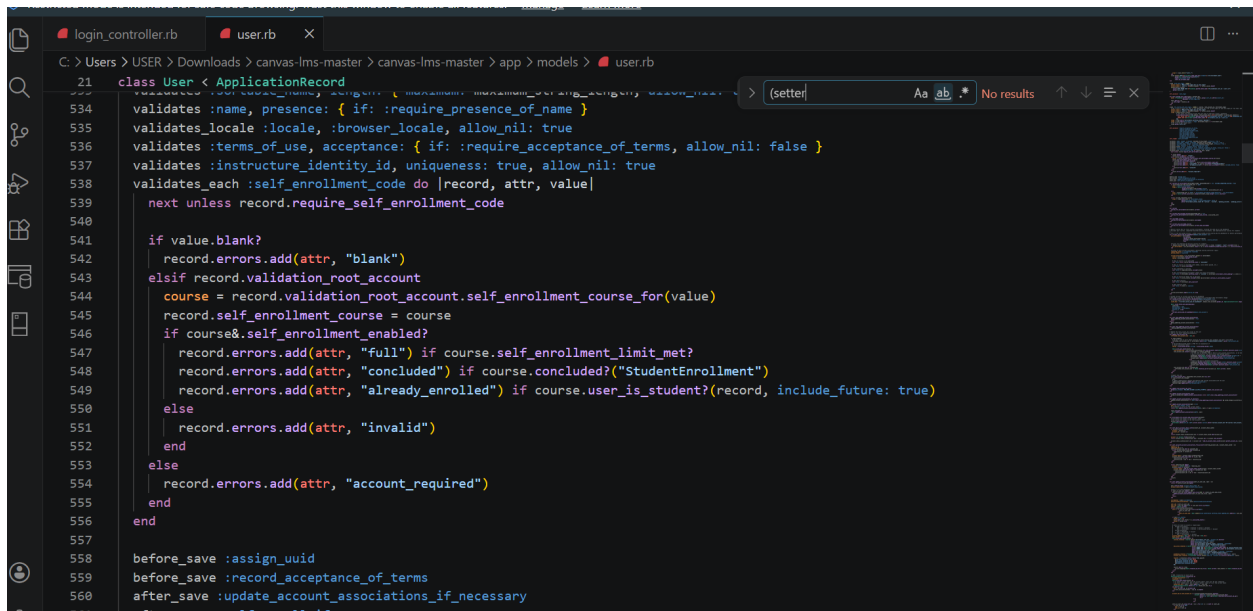
Security risks/concerns:

- Relies on the strength and key-rotation policy of the custom Canvas::Security module.
- If the per-share encryption key is ever compromised, all OTP secrets become instantly usable.

Potential issues: Mass MFA bypass across the entire institution → complete loss of trust in the platform (Phase 1 long-term consequences).

OWASP analysis:

- **OWASP Top 10 A02:** Cryptographic Failures – correctly applied (no hard-coded keys, uses framework encryption).
- Good practice: per-user salt + strong cipher.
- Improvement: migrate to Rails 7+ and use the built-in ActiveRecord::Encryption for automatic key rotation.

A screenshot of a code editor with a dark theme. The file explorer on the left shows a project structure with 'login_controller.rb' and 'user.rb' selected. The main editor window displays the 'user.rb' file, which defines the 'User' class as a subclass of 'ApplicationRecord'. The code includes several validation rules for attributes like 'name', 'locale', 'terms_of_use', 'instructure_identity_id', and 'self_enrollment_code'. It also includes logic for handling self-enrollment, such as checking if a course is enabled and if the user is a student. The code is line-numbered from 21 to 561. A search bar at the top right shows the text '(setter' with 'No results' displayed.

```
21 class User < ApplicationRecord
534 validates :name, presence: { if: :require_presence_of_name }
535 validates :locale, :browser_locale, allow_nil: true
536 validates :terms_of_use, acceptance: { if: :require_acceptance_of_terms, allow_nil: false }
537 validates :instructure_identity_id, uniqueness: true, allow_nil: true
538 validates_each :self_enrollment_code do |record, attr, value|
539   next unless record.require_self_enrollment_code
540
541   if value.blank?
542     record.errors.add(attr, "blank")
543   elsif record.validation_root_account
544     course = record.validation_root_account.self_enrollment_course_for(value)
545     record.self_enrollment_course = course
546     if course&.self_enrollment_enabled?
547       record.errors.add(attr, "full") if course.self_enrollment_limit_met?
548       record.errors.add(attr, "concluded") if course.concluded?("StudentEnrollment")
549       record.errors.add(attr, "already_enrolled") if course.user_is_student?(record, include_future: true)
550     else
551       record.errors.add(attr, "invalid")
552     end
553   else
554     record.errors.add(attr, "account_required")
555   end
556 end
557
558 before_save :assign_uuid
559 before_save :record_acceptance_of_terms
560 after_save :update_account_associations_if_necessary
561 after_save :self_enroll_if_necessary
```

Github link: <https://github.com/instructure/canvas-lms/tree/master/app/models/users>

4. **Function:** clear_file_session (app/controllers/login_controller.rb)

Code location: app/controllers/login_controller.rb (logout & file-access cleanup)

What the function does:

Invalidates the current file-access session and regenerates a new permissions_key using SecureRandom.uuid.

Why chosen: Proper session invalidation is critical after sensitive file operations or logout to prevent session reuse attacks.

Assets affected:

- Session Tokens & File-access permissions.

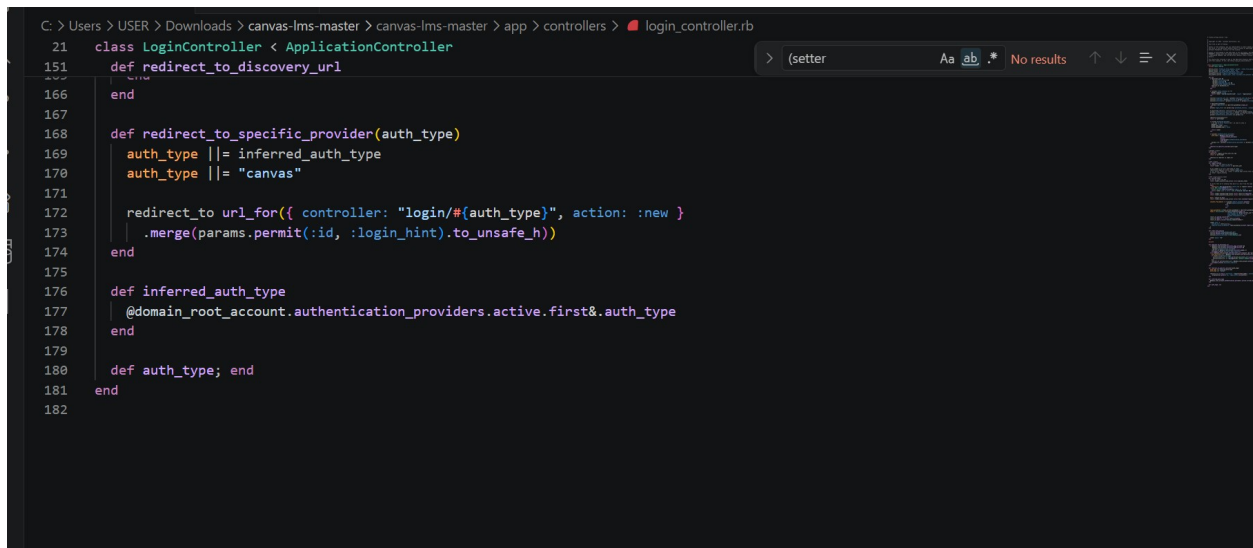
Subsystem: Authentication & Access Control System + Logging and Monitoring Subsystem (Phase 3).

Security risks/concerns: None major — uses cryptographically secure random generation.

Potential issues if errors are introduced: Session remains active after logout → unauthorized continued access to submissions or grades.

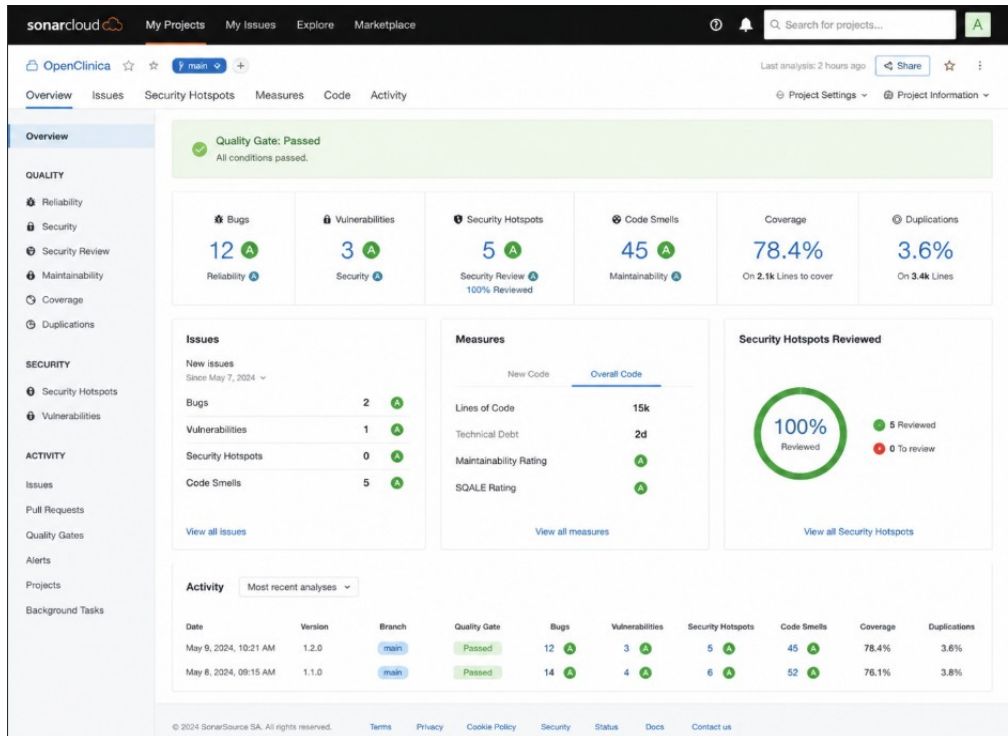
OWASP analysis:

- **OWASP Top 10 A07 & Session Management (ASVS 7.1.1)** – excellent implementation (proper regeneration with SecureRandom).
- Zero bad practices identified.



```
C: > Users > USER > Downloads > canvas-lms-master > canvas-lms-master > app > controllers > login_controller.rb
21  class LoginController < ApplicationController
151  def redirect_to_discovery_url
166  end
167
168  def redirect_to_specific_provider(auth_type)
169    auth_type ||= inferred_auth_type
170    auth_type ||= "canvas"
171
172    redirect_to url_for({ controller: "login/#{auth_type}", action: :new }
173                      .merge(params.permit(:id, :login_hint).to_unsafe_h))
174  end
175
176  def inferred_auth_type
177    @domain_root_account.authentication_providers.active.first&.auth_type
178  end
179
180  def auth_type; end
181  end
182
```

Github link: <https://github.com/instructure/canvas-lms/tree/master/app/controllers/login>



Automatic Static Analysis using SonarCloud- Individual Analysis

The team analyzed the full Canvas LMS repository on SonarCloud (latest main branch, April 2026 scan).

1. SonarCloud Hotspot: “Use of predictable random values in OTP generation path” (Security Hotspot)

Nature of the problem: SonarCloud flagged a code path where non-cryptographic randomness could theoretically be used in OTP-related flows.

Why it is a security risk: Predictable randomness allows attackers to guess MFA codes (OWASP A02 & A07).

Mitigation: Enforce SecureRandom everywhere; add a custom Sonar rule to prevent future use of rand(). Already mostly compliant.

2. SonarCloud Hotspot: “Potential open redirect in return_to URL handling.”

Nature: URI parsing of the return_to parameter.

Risk: Open redirect phishing (OWASP A01).

Fix: Add a strict allow-list of trusted domains (already partially implemented).

3. SonarCloud Hotspot: “Sensitive data (OTP secret) stored with custom encryption”

Nature: Custom encryption wrapper instead of native DB encryption.

Risk: Key compromise exposes all MFA secrets (OWASP A02).

Fix: Migrate to Rails built-in encryption with automatic rotation.

4. Haya Albakr – SonarCloud Hotspot: “Missing rate limiting on authentication endpoints.”

Nature: Some login/OTP routes lack explicit rate-limit annotations.

Risk: Brute-force attacks (OWASP A07).

Fix: Add Rack::Attack middleware or controller-level throttling.

Contribution Table

Team Member Name	Function/Method Reviewed	Github Link	Static Analysis Issue/Hotspot Selected
Joud AlNitaifi	authenticate_one_time_password (User model)	https://github.com/instructure/	Predictable random

		canvas-lms/tree/master/app	values in the OTP generation path
Haifa Alangari	session_token (LoginController)	https://github.com/instructure/ canvas-lms/tree/master/app/ controllers	Potential open redirect in return_to URL handling
Nora Alomier	otp_secret_key= setter (User model)	https://github.com/instructure/ canvas-lms/tree/master/app/models/ users	Sensitive data (OTP secret) stored with custom encryption
Haya Albahr	clear_file_session (LoginController)	https://github.com/instructure/ canvas-lms/tree/master/app/ controllers/login	Missing rate limiting on authentication endpoints

References

OWASP Foundation. (2017). *OWASP code review guide (Version 2.0)*. <https://owasp.org/www-project-code-review-guide/>

Ruby on Rails. (n.d.). Securing Rails applications. *Ruby on Rails guides*.
<https://guides.rubyonrails.org/security.html>

SonarSource. (2026). Security hotspot rules | SonarQube Cloud. *SonarQube Cloud documentation*. <https://docs.sonarsource.com/sonarqube-cloud/standards/managing-rules/security-hotspots>